

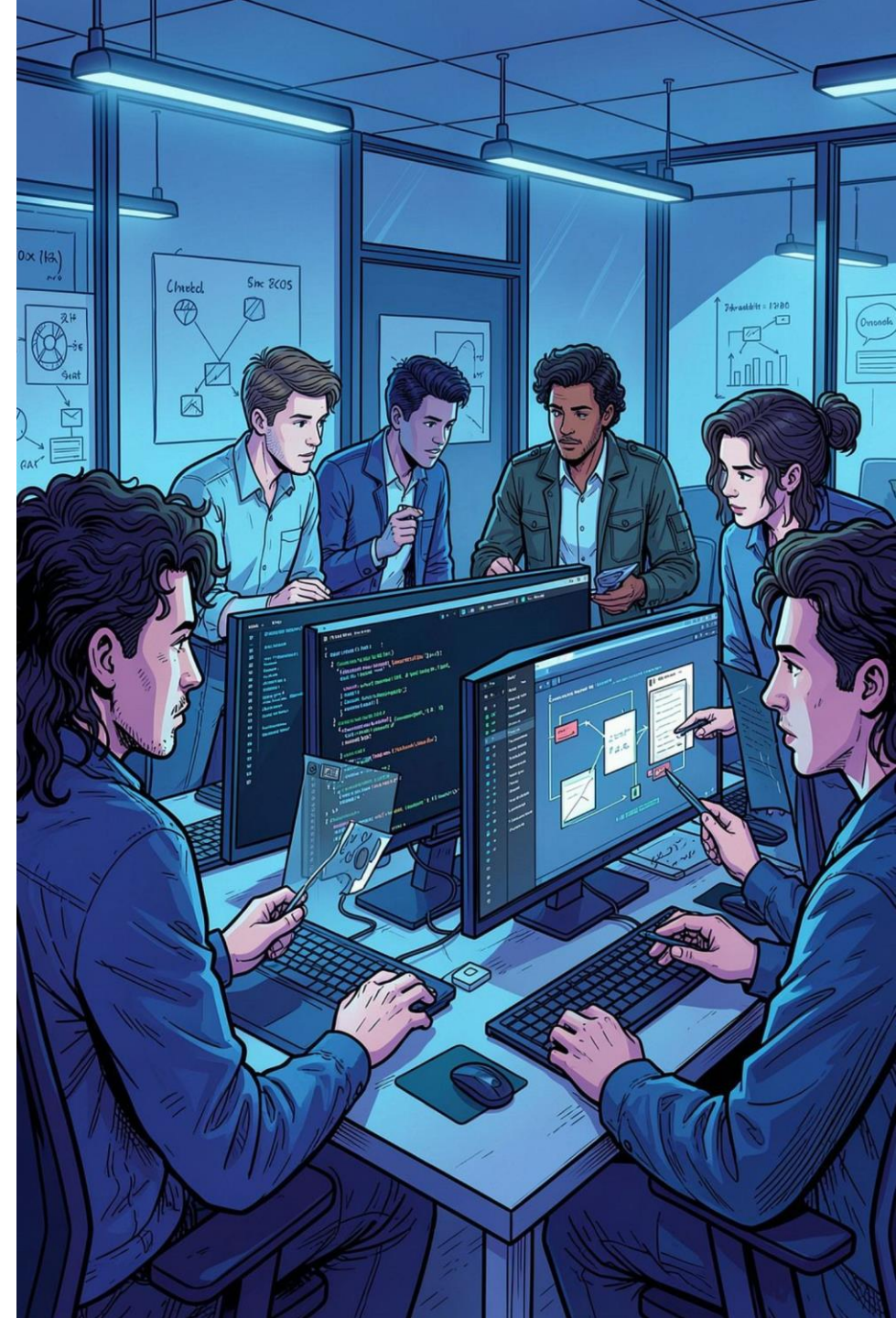
ENGENHARIA DE SOFTWARE

AULA 3

PROF. ME. VALDIR PRADO

Processos de Engenharia de Software

As atividades metodológicas e os fluxos de trabalho que fazem o software rodar de verdade.



O Motor da Engenharia de Software

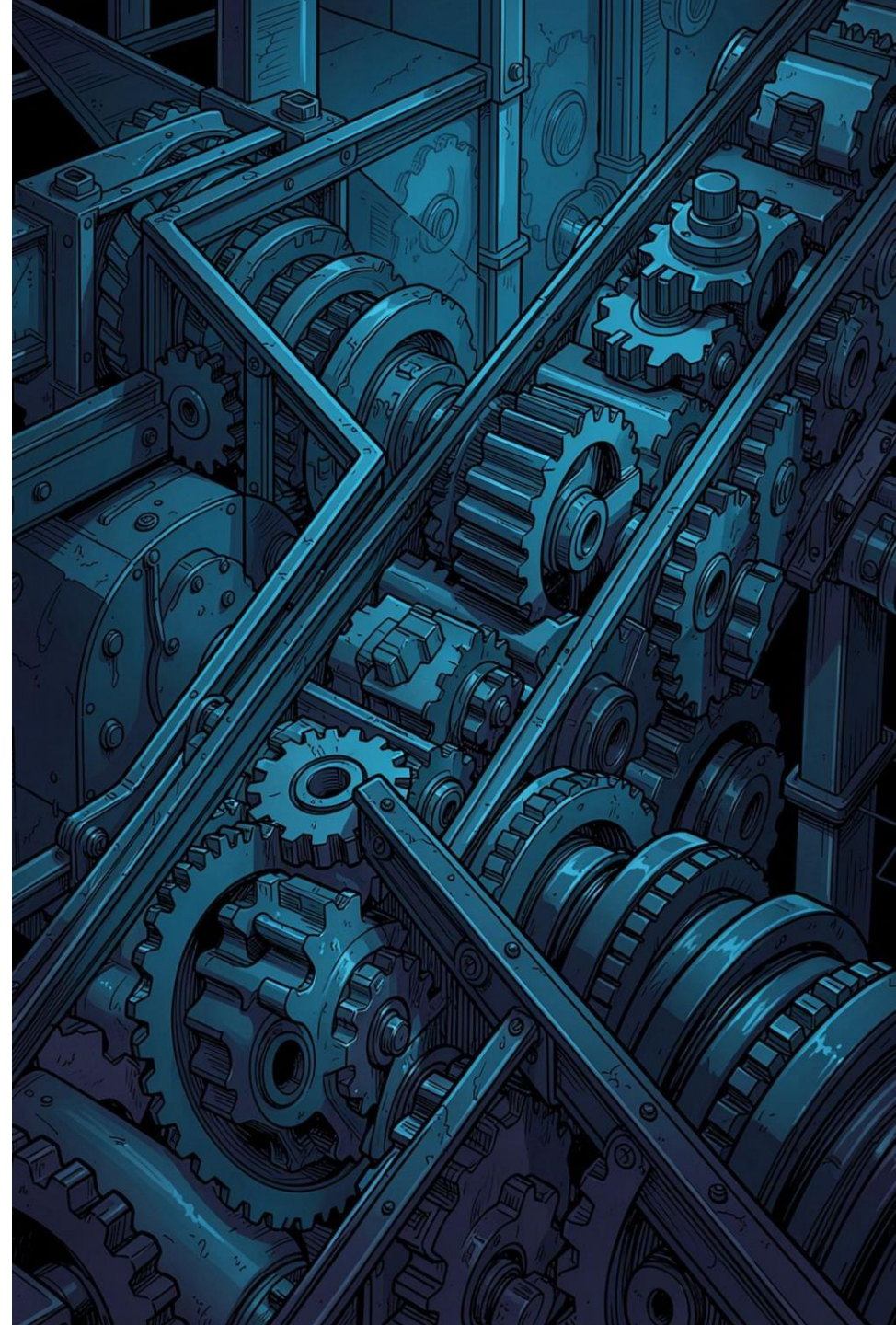
Um **processo de software** é um conjunto de atividades relacionadas que levam à produção de um produto de software. Em qualquer empresa de tecnologia, quatro atividades fundamentais são inegociáveis.

Especificação

Projeto e Implementação

Validação & Verificação

Evolução



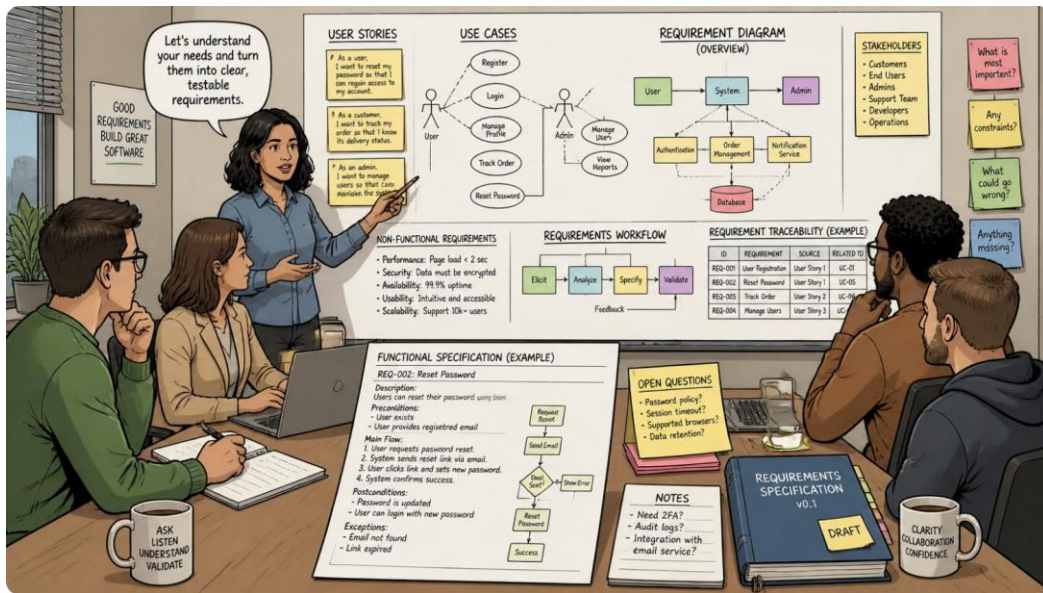
Especificação ou Engenharia de Requisitos

Engenharia de Requisitos

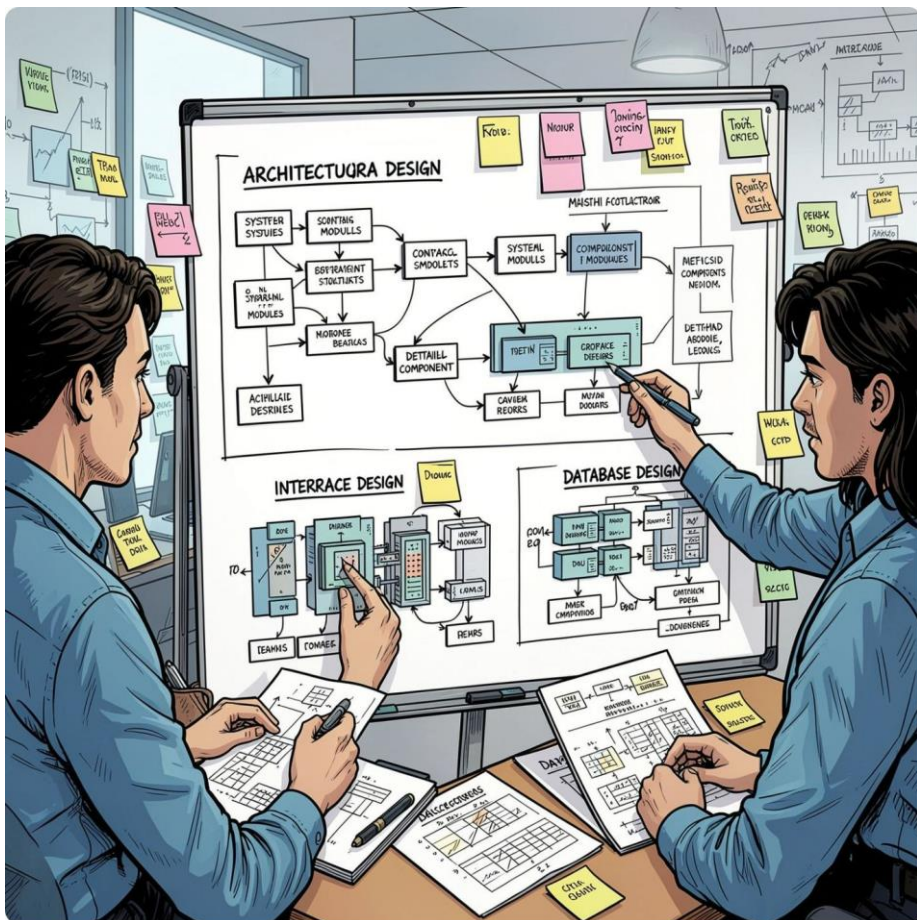
Define **o que** o software deve fazer e **em quais condições** deve operar. Uma especificação bem feita reduz riscos, retrabalho e custos.

- **Viabilidade:** avalia custo, benefício e restrições do projeto.
- **Elicitação e análise:** identifica necessidades por meio de conversas e levantamento estruturado.
- **Especificação:** documenta requisitos de usuário e de sistema com precisão.
- **Validação:** detecta conflitos, omissões e ambiguidades antes do desenvolvimento.

Erros na especificação comprometem todo o projeto.



O Projeto de Software (Design) e Implementação



Projeto e Implementação

Converte a especificação em sistema executável através de visões claras de modelagem:

- **Projeto de Arquitetura:** define a estrutura geral do sistema e seus componentes principais
- **Projeto de Interfaces:** especifica como os componentes se comunicam entre si
- **Projeto Detalhado:** detalha a estrutura interna de cada componente
- **Projeto de Bancos de Dados:** modela as estruturas de dados e relacionamentos

A Implementação traduz tudo isso em código real.

Verificação e Validação (V&V) na Prática

O processo geral de V&V visa estabelecer a confiança de que o software é adequado ao seu propósito e está pronto para uso operacional. A distinção clássica de Barry Boehm dita o norte das atividades:



Verificação

Avalia se o sistema está em conformidade com as suas especificações técnicas e requisitos funcionais e não funcionais.

"Estamos construindo o produto da maneira certa?"



Validação

Avalia se o sistema atende às reais expectativas operacionais e necessidades do cliente (indo além da simples especificação formal, que pode conter omissões).

"Estamos construindo o produto certo?"

Na prática da engenharia de software, as atividades de V&V dividem-se em dois grandes pilares complementares, pois nenhuma técnica isolada é capaz de detectar todos os defeitos.

Verificação e Validação (V&V) na Prática

As atividades de V&V dividem-se em dois grandes pilares complementares, pois nenhuma técnica isolada é capaz de detectar todos os defeitos do software.



V&V Estático (Revisões, Inspeções e Análises)

Nesta abordagem, o software **não é executado**. É uma técnica de altíssimo custo-benefício, aplicável em documentos de requisitos, diagramas UML, planos de testes e código-fonte.

- **Inspeções de Programa (Peer Reviews):** Revisões em pares onde engenheiros examinam detalhadamente o código-fonte à procura de defeitos lógicos, falhas de padronização ou vulnerabilidades.
- **Análise Estática Automatizada:** Ferramentas examinam o texto do programa, identificando anomalias estruturais como variáveis não usadas, possíveis overflows de buffer ou loops sem saída.

Verificação e Validação (V&V) na Prática

As atividades de V&V dividem-se em dois grandes pilares complementares, pois nenhuma técnica isolada é capaz de detectar todos os defeitos do software.

V&V Dinâmico (Testagem do Sistema)

Consiste em **executar o software** utilizando dados de teste para observar o seu comportamento. Na prática industrial, o teste é orientado pelo Modelo V, correlacionando fases de desenvolvimento com níveis de testagem.



- **Testes Unitários:** Desenvolvedor foca na menor unidade de código (classes/métodos), isolando-as de suas dependências.
- **Testes de Integração:** Testa grupos de componentes inter-relacionados para garantir interfaces coesas e detectar comportamentos emergentes.
- **Testes de Release (Caixa-Preta):** Equipe independente valida funcionalidade e desempenho contra requisitos especificados.
- **Testes de Aceitação:** Estágio final onde o cliente testa o sistema completo em seu ambiente operacional real para decidir sobre a implantação.

Evolução — O Ciclo Infinito

Evolução

O mercado atual exige que o software evolua constantemente para atender às necessidades mutáveis dos clientes, tornando-se um processo contínuo. Historicamente, desenvolvimento e manutenção eram separados.

O ciclo de vida de evolução de Rajlich e Bennett:

- **desenvolvimento inicial;**
- **evolução;**
- **em serviço;**
- **interrupção gradual**

Entram também, as **Leis de Lehman**, especialmente a primeira, que estabelece que a mudança do software é um processo inevitável para que ele continue útil em seu ambiente operacional



As Leis de Lehman: A Dinâmica da Evolução de Software

Compreender que o software não é estático é um dos maiores saltos de maturidade para um aluno de engenharia. Dois terços a 90 % dos custos organizacionais de software são custos de evolução pós-implantação. Manny Lehman e Les Belady formularam as Leis de Lehman para explicar essa realidade em sistemas de grandes organizações (E-type):

1. Mudança Contínua

Um programa em ambiente real deve necessariamente mudar, ou se tornará progressivamente menos útil. A reintrodução gera novas demandas, criando um ciclo infinito de evolução.

2. Aumento de Complexidade

À medida que o programa evolui, sua estrutura tende a se degradar e se tornar mais complexa. Recursos devem ser dedicados à manutenção preventiva e refatoração.

3. Evolução de Programa de Grande Porte (Autorregulação)

A evolução de sistemas de grande escala é um processo autorregulador. Atributos como tamanho, tempo entre lançamentos e erros mantêm-se aproximadamente constantes.

4. Estabilidade Organizacional

A taxa média de desenvolvimento é aproximadamente constante e independente dos recursos adicionais. Equipes saturadas sofrem com overhead de comunicação.

As Leis de Lehman: A Dinâmica da Evolução de Software

Compreender que o software não é estático é um dos maiores saltos de maturidade para um aluno de engenharia. Dois terços a 90% dos custos organizacionais de software são custos de evolução pós-implantação. Manny Lehman e Les Belady formularam as Leis de Lehman para explicar essa realidade em sistemas de grandes organizações (E-type):

5. Conservação da Familiaridade

A mudança incremental em cada versão é aproximadamente constante. Funcionalidade excessiva introduz defeitos que forçam releases de estabilização.

6. Crescimento Contínuo

A funcionalidade oferecida precisa aumentar continuamente para manter a satisfação dos usuários.

7. Declínio de Qualidade

A qualidade percebida cairá constantemente, a menos que o software seja ativamente modificado e adaptado ao seu ambiente operacional.

8. Sistema de Feedback

Os processos de evolução incorporam sistemas de feedback multiagentes e multiloop de longo prazo que precisam ser gerenciados sistematicamente.

O Paradoxo dos Requisitos Voláteis e como Testá-los

Os requisitos voláteis, comuns em sistemas complexos, são aqueles que mudam constantemente devido a pressões de mercado, mudanças organizacionais, novas legislações ou ajustes internos. Para construir um produto resiliente a essa volatilidade, a engenharia de software adota táticas rigorosas de testabilidade e gerenciamento de mudanças.

1

Verificabilidade (Testabilidade)

Requisitos devem ser escritos de forma quantitativa e objetiva, permitindo a criação de casos de teste específicos para cada critério. Requisitos não testáveis precisam ser reformulados.

2

Rastreabilidade (Traceability)

Manter registros explícitos que vinculem requisitos a casos de teste e dependências. Isso é crucial para avaliar o impacto real e o custo de qualquer alteração proposta.

3

Test-First Development

Escrever a infraestrutura de testes automatizados antes de implementar o código funcional. Isso força a equipe a debugar e consolidar o entendimento dos requisitos desde o início.

O Paradoxo dos Requisitos Voláteis e como Testá-los

Os requisitos voláteis, comuns em sistemas complexos, são aqueles que mudam constantemente devido a pressões de mercado, mudanças organizacionais, novas legislações ou ajustes internos. Para construir um produto resiliente a essa volatilidade, a engenharia de software adota táticas rigorosas de testabilidade e gerenciamento de mudanças.

4

Testes de Regressão Automatizados

Com a automação (ex: JUnit), um conjunto completo de testes anteriores pode ser executado instantaneamente a cada alteração, assegurando que o novo código não introduza bugs em funcionalidades estáveis.

5

Testes de Aceitação Incrementais

O cliente participa ativamente, definindo e executando testes de aceitação baseados em histórias de usuário. Rodados incrementalmente, validam que o software em constante mudança atende às demandas do negócio.

6

Uso de Prototipagem

Desenvolvimento ágil de protótipos de interface permite que o usuário interaja e valide suas necessidades em estágios iniciais, antecipando ambiguidades e prevenindo mudanças dispendiosas pós-entrega.

O Framework RUP (Rational Unified Process)

O Rational Unified Process (RUP) é um framework **híbrido e maduro** que integra as melhores práticas da engenharia de software convencional e incremental. Ele aborda a complexidade do desenvolvimento sob **três perspectivas cruciais** para garantir a organização e a adaptabilidade do processo:

1

Perspectiva Dinâmica (Temporal)

Foca nas metas de negócio, dividindo o processo em **quatro grandes fases**: Concepção, Elaboração, Construção e Transição, que guiam o projeto temporalmente.

Nas **fases dinâmicas iniciais** (*Concepção e Elaboração*), a equipe concentra a maior parte do esforço em workflows como **Modelagem de Negócios e Requisitos**

Nas **fases dinâmicas posteriores** (*Construção e Transição*), o esforço da equipe migra drasticamente para os workflows de **Implementação, Teste e Implantação**

O Framework RUP (Rational Unified Process)

2

Perspectiva Estática (Técnica)

Organiza o trabalho em **workflows** (fluxos de trabalho), detalhando as atividades e responsabilidades técnicas de diversas disciplinas da engenharia de software.

3

Perspectiva Prática (Boas Práticas)

Define essenciais para o sucesso, promovendo resiliência e qualidade:

- Desenvolvimento iterativo
- Gerenciamento rigoroso de requisitos
- Arquitetura baseada em componentes
- Modelagem visual com a UML
- Verificação rigorosa da qualidade
- Controle estrito de mudanças com gerência de configuração

O Framework RUP

O **Rational Unified Process (RUP)** organiza a complexidade do desenvolvimento reconhecendo que modelos convencionais falham por apresentar uma visão única do processo. É estruturado em três perspectivas complementares.



A Integração no RUP: Um Processo Contínuo

No Rational Unified Process (RUP), a integração não é uma fase isolada, mas uma atividade contínua e fundamental que permeia a fase de Construção e se desdobra em diversos fluxos de trabalho. É aqui que as partes desenvolvidas em paralelo convergem para formar um sistema funcional.



Fase de Construção

Envolve projeto, programação e testes. As diversas partes são desenvolvidas em paralelo e integradas para gerar um software funcional pronto para entrega.



Workflow de Implementação

Componentes individuais são estruturados em subsistemas de implementação, garantindo a coesão e o acoplamento correto das unidades de software.



Workflow de Teste

Um processo iterativo realizado lado a lado com a implementação, seguido pelo teste do sistema integrado para validar a funcionalidade e o desempenho.



A "**Integração e teste de sistema**" é uma fase explícita no clássico **Modelo em Cascata (Waterfall)**, onde os componentes são integrados e testados sequencialmente, apenas após a conclusão de toda a codificação. No RUP, é um processo dinâmico e iterativo.

Visão Estática: Fluxos de Trabalho

Fluxos Centrais de Engenharia

- Modelagem de Negócios
- Requisitos
- Análise e Projeto
- Implementação
- Teste
- Implantação

Fluxos de Apoio

Gerenciamento de
Configuração

Gerenciamento de Projeto

Meio Ambiente

📄 O diferencial do RUP: todos os fluxos podem estar **ativos simultaneamente** em qualquer fase do projeto.

Missão Prática: Mapa Mental

Turma, agora vocês assumem a cadeira de **arquitetos de processos**. Vocês têm **55 minutos** para desenhar um mapa mental colaborativo detalhando a esteira de desenvolvimento de um novo sistema de controle logístico.

3 Grupos

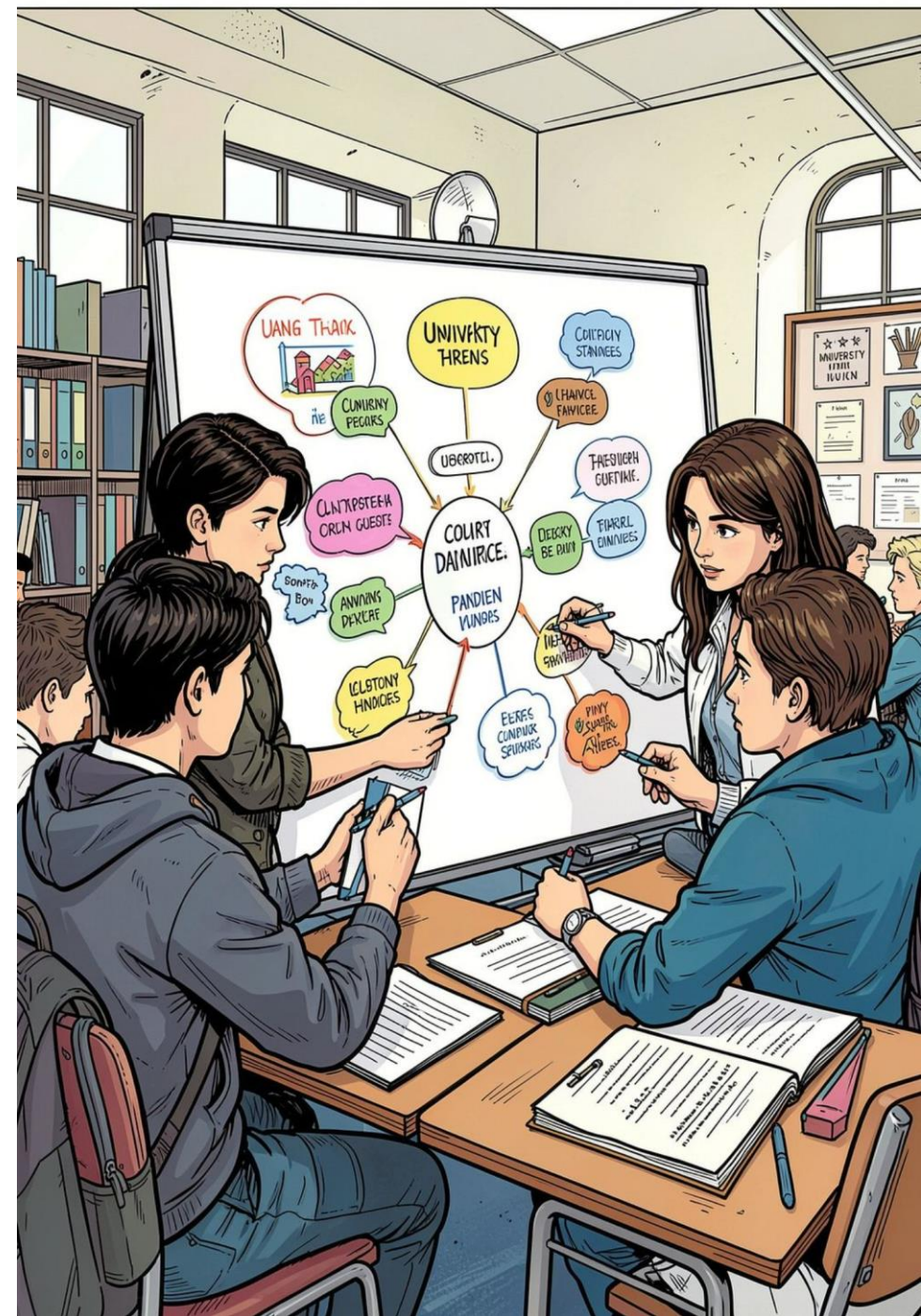
2 grupos de 5 pessoas + 1 grupo de 4 pessoas (total: 14)

Objetivo

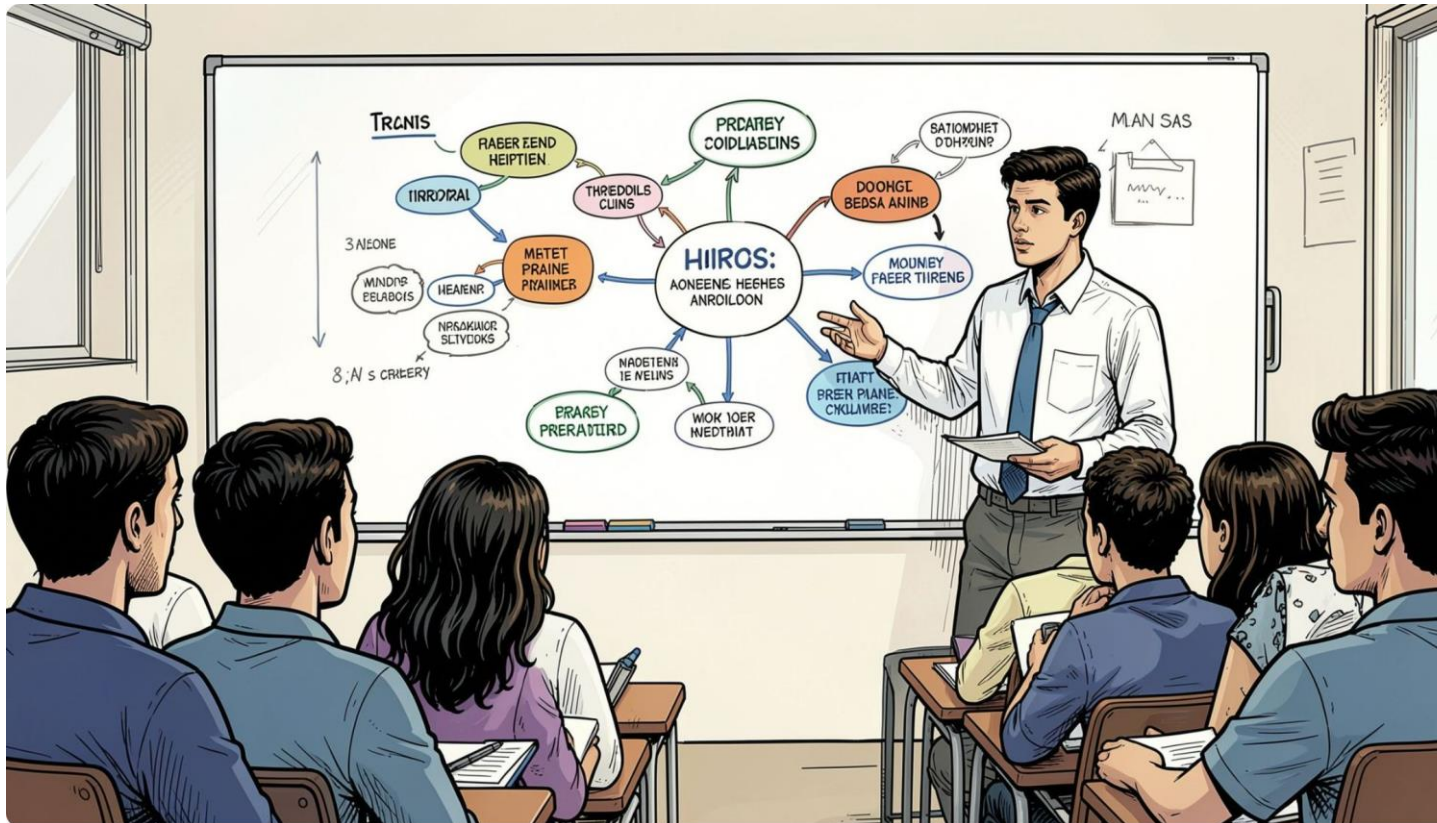
Correlacionar as 4 atividades fundamentais com os fluxos centrais do RUP

Foco

Como "Análise e Projeto" e "Teste" interagem nas fases de Elaboração (mitigação de riscos e arquitetura) e Construção (programação paralela e integração) de um **Sistema de Controle Logístico Multimodal** (rastreamento de frotas, alocação de inventário de armazém e cálculo inteligente de fretes)



Defesa dos Fluxos Estruturados



O que avaliar

A clareza com que vocês integram o fluxo de requisitos e a validação dentro das fases do RUP é o que determina se a esteira da empresa será **ágil ou burocrática**.

Por que importa

Uma modelagem visual bem executada **elimina os gargalos de comunicação** entre os times de negócio e de programação.

Fechamento Técnico

A Engenharia de Software moderna não separa a teoria da prática; a forma como organizamos as atividades metodológicas define o sucesso financeiro e técnico de qualquer projeto de tecnologia.



Atividades Fundamentais

Especificação, Projeto, Validação e Evolução formam o núcleo de qualquer esteira de software.



RUP e Workflows

Os fluxos de trabalho simultâneos do RUP organizam a complexidade do desenvolvimento em três perspectivas.



Próxima Aula

Esses fluxos serão a base das nossas próximas análises práticas.

5. REFERÊNCIAS BIBLIOGRÁFICAS

- **SOMMERVILLE, Ian.** *Engenharia de Software*. 10. ed. São Paulo: Pearson Education, 2016. (Capítulo 2: Processos de Software; Capítulo 4: Engenharia de Requisitos; Capítulo 8: Teste de Software; Capítulo 9: Evolução de Software).
- **PRESSMAN, Roger S.; MAXIM, Bruce R.** *Engenharia de Software: Uma Abordagem Profissional*. 9. ed. Porto Alegre: AMGH, 2021. (Capítulo 3: Modelos de Processo de Software; Capítulo 4: Engenharia de Requisitos; Capítulo 15: Verificação e Validação).
- **LEHMAN, M. M.** Laws of Software Evolution Revisited. *Proceedings of the European Workshop on Software Process Technology (EWSPT'96)*. Springer-Verlag, 1996, p. 108-124.
- **BOEHM, Barry W.** *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice Hall, 1981. (Princípios Fundamentais de Verificação e Validação de Software).